

01-406-093-203

การพัฒนาเว็บแอปพลิเคชันด้วยเฟรมเวิร์ค

โดย นายศุภมิตร จันทรโทวงศ์

- สาขาวิชาระบบสารสนเทศ คณะบริหารธุรกิจ

เช็คชื่อเข้าชั้นเรียน

- ให้นักศึกษาสแกน QR Code นี้เพื่อเข้าสู่ระบบเช็คชื่อเข้าเรียน
- ระบบจะทำการบันทึกประวัติการเข้าเรียนในวิชานี้อัตโนมัติ

สแกนเพื่อเช็คชื่อเข้าชั้นเรียน (สัปดาห์ที่ 12)



สแกน QR Code เพื่อเช็คชื่อเข้าเรียนของสัปดาห์นี้

แบบทดสอบก่อนเรียน (Pre-test)

1. คำสั่ง Composer ใดที่ใช้ในการติดตั้งแพ็คเกจสำหรับสร้างระบบ Authentication พื้นฐานใน Laravel?

A. `composer require laravel/ui`

B. `composer require laravel/auth`

C. `composer install laravel/login`

D. `php artisan make:auth`

3. หลังจากรันคำสั่งสร้าง Scaffolding ของ UI แล้ว ต้องรันคำสั่งใดเพื่อดาวนโหลด Node Dependencies (เช่น Bootstrap และ Sass)?

A. `npm install`

B. `composer update`

C. `npm start`

D. `php artisan migrate`

5. ไฟล์ Master Layout หลักที่ระบบ Authentication สร้างขึ้นให้อัตโนมัติถูกจัดเก็บไว้ที่พาทใด?

A. `resources/views/layouts/app.blade.php`

B. `resources/views/master.blade.php`

C. `app/Views/layouts/main.blade.php`

D. `public/views/app.blade.php`

7. ในส่วน Navbar ของ `layouts/app.blade.php` หากต้องการแสดงชื่อของผู้ใช้งานที่กำลังล็อกอินอยู่ ต้องใช้คำสั่งใด?

A. `{{ Auth::user()->name }}`

B. `{{ User::getName() }}`

C. `{{ Session::get('user') }}`

D. `{{ $user_name }}`

9. หน้า View ลูก (เช่น `blog.blade.php`) ต้องใช้คำสั่งใดที่บรรทัดบนสุดเพื่อสืบทอดโครงสร้างและ Navbar จาก `app.blade.php`?

A. `@extends('layouts.app')`

B. `@include('layouts.app')`

C. `@import('layouts.app')`

D. `@use('layouts.app')`

2. คำสั่ง Artisan ใดที่ใช้สร้างหน้าจอ Login, Register และ Controller ที่เกี่ยวข้องด้วย Bootstrap?

A. `php artisan ui bootstrap --auth`

B. `php artisan make:auth --bootstrap`

C. `php artisan auth:scaffold bootstrap`

D. `php artisan ui:auth --bootstrap`

4. คำสั่งใดที่ใช้ให้ Vite ทำการคอมไพล์ไฟล์ SCSS, CSS และ JavaScript สำหรับการใช้งานจริง (Production Build)?

A. `npm run build`

B. `npm run dev`

C. `npm compile`

D. `npm run prod`

6. ใน Blade Template แกกคำสั่งใดที่ใช้ครอบการแสดงผลเนื้อหาเฉพาะสำหรับผู้ใช้งานที่ยังไม่ได้ล็อกอิน (Guest)?

A. `@guest ... @endguest`

B. `@auth ... @endauth`

C. `@user ... @enduser`

D. `@visitor ... @endvisitor`

8. หากต้องการให้แต่ละหน้า View สามารถกำหนดชื่อ Title ของตนเองเพื่อแสดงบนแท็บบน Browser ได้ ใน `layouts/app.blade.php` ควรกำหนดอย่างไร?

A. `<title>@yield('title')</title>`

B. `<title>{{ $title }}</title>`

C. `<title>@include('title')</title>`

D. `<title>@section('title')</title>`

10. หากต้องการป้องกันไม่ให้ผู้ใช้ที่ยังไม่ได้ล็อกอินเข้าถึงทุก Method ใน Controller (เช่น `HomeController` หรือ `AdminController`) ต้องกำหนดคำสั่งใดใน `__construct()`?

A. `$this->middleware('auth');`

B. `$this->protect('login');`

C. `$this->checkAuth();`

D. `Auth::check();`

สัปดาห์ที่ 12 · ระบบยืนยันตัวตนและการเข้าถึง

01-406-093-203 LARAVEL COURSE

Authentication (ระบบลงชื่อเข้าใช้งาน)

อาจารย์ผู้บรรยาย: ศุภมิตร จันทโรวงค์

สไลด์ที่ 4 / 23

ภาพรวมโปรเจกต์ (Project Overview)

ภาพรวมโปรเจกต์ (Project Overview)

ภาพรวมโปรเจกต์	
กลุ่มผู้ใช้งานระบบ	ความสามารถ
นักเขียน (Author)	จัดการข้อมูลบทความได้ - เพิ่ม , ลบ , แก้ไขบทความ - เผยแพร่ / ฉบับร่าง บทความ
ผู้ใช้งาน (User)	อ่านบทความที่เผยแพร่ได้

การติดตั้ง Package สำหรับ Authentication (laravel/ui)

- เปิด Terminal ในโปรเจกต์ Laravel (เช่น ใน VS Code Terminal)
- รับคำสั่ง `composer require laravel/ui` เพื่อดาวน์โหลดและติดตั้ง Laravel UI Package
- Laravel UI เป็นเครื่องมือช่วยสร้าง Scaffolding สำหรับระบบ Authentication พื้นฐาน
- เตรียมความพร้อมสำหรับการสร้างหน้า Login, Register และการจัดการผู้ใช้

code-example.bash

```
composer require laravel/ui
```

การสร้างระบบ Authentication ด้วย Bootstrap (`php artisan ui bootstrap --auth`)

- รับคำสั่ง `php artisan ui bootstrap --auth` ใน Terminal เพื่อสร้างระบบ Login/Register แบบ Bootstrap
- ระบบจะสร้าง View สำหรับ Login, Register, Password Reset และ Controller ที่เกี่ยวข้องให้อัตโนมัติ
- หากมีข้อความถามการ Replace ไฟล์ `Controller.php` ให้พิมพ์ `yes` แล้วกด Enter
- ระบบจะสร้าง Scaffolding ของ Authentication และโครงสร้างหน้า Layout พื้นฐานให้เสร็จสมบูรณ์

code-example.bash

```
php artisan ui bootstrap --auth
```

การติดตั้ง Node Dependencies (`npm install`)

- รับคำสั่ง `npm install` เพื่อดาวนิโกลดและติดตั้ง JavaScript/CSS Packages ของระบบ (เช่น Bootstrap & Sass)
- อ่านการตั้งค่าจากไฟล์ `package.json` ที่ถูกอัปเดตหลังจากสร้าง UI scaffolding
- ระบบจะดาวนิโกลด dependencies ที่จำเป็นลงในโฟลเดอร์ `node_modules`
- เป็นการเตรียมพร้อมสำหรับการ compile ไฟล์ asset หน้าบ้านในขั้นตอนถัดไป

code-example.bash

```
npm install
```


การคอมไพล์และ Build ไฟล์ Assets (`npm run build`)

- รับคำสั่ง `npm run build` เพื่อให้ Vite ทำการคอมไพล์ไฟล์ SCSS, CSS และ JavaScript
- ระบบจะสร้างไฟล์ Asset สำหรับใช้งานจริง (Production Build) ลงในโฟลเดอร์ `public/build`
- สร้างไฟล์ `manifest.json` , `app.css` และ `app.js` ที่ผูกกับระบบ Bootstrap ให้พร้อมแสดงผล
- ช่วยให้หน้า Login/Register และหน้า Layout แสดงสไตล์ Bootstrap ได้อย่างสมบูรณ์สวยงาม

code-example.bash

```
npm run build
```

การปรับแต่ง Layout สำหรับผู้ใช้งาน (@auth & @guest)

ไฟล์ Layout หลัก (`resources/views/layouts/app.blade.php`)

- ไฟล์โครงสร้างหน้าหลัก (Master Layout) สำหรับระบบ Authentication
- จัดเก็บอยู่ที่ `resources/views/layouts/app.blade.php`
- ประกอบด้วยโครงสร้าง HTML พื้นฐาน, ส่วนหัว Header/Navbar และจุดเชื่อมโยง `@yield('content')`
- หน้าอื่นๆ สามารถเรียกใช้ผ่านคำสั่ง `@extends('layouts.app')` ได้ทันที

code-example.php

```
// เส้นทางไฟล์ Layout หลักของระบบ Authentication
resources/
├── views/
│   └── layouts/
│       └── app.blade.php ★ (Master Layout ของระบบ Auth)
```

การกำหนดชื่อหัวข้อหน้าเว็บ (`<title>@yield('title')</title>`)

- แก้ไขแก้ไข `<title>` ภายในไฟล์ `resources/views/layouts/app.blade.php`
- ใช้คำสั่ง `@yield('title')` เพื่อเปิดช่องให้ View ลูกส่งชื่อหัวข้อหน้าเข้ามาแสดง
- หน้าอื่นๆ กำหนดชื่อหัวข้อได้ง่ายๆ ด้วย `@section('title', 'ชื่อหัวข้อ')`
- ช่วยให้ชื่อแท็บของเบราว์เซอร์เปลี่ยนตามแต่ละหน้าเว็บได้อย่างถูกต้อง

code-example.html

```
<!-- resources/views/layouts/app.blade.php -->  
<title>@yield('title')</title>
```

การปรับแต่งแถบเมนูใน Layout สำหรับผู้มาเยือน (@guest)

- เปิดไฟล์ `resources/views/layouts/app.blade.php` ที่ระบบสร้างขึ้นอัตโนมัติ
- ค้นหาส่วน `<!-- Right Side Of Navbar -->` ภายใต้แท็ก `@guest`
- แก้ไขข้อความปุ่ม Login เป็น `เข้าสู่ระบบ`
- แก้ไขข้อความปุ่ม Register เป็น `สมัครเป็นนักเขียน` เพื่อให้ตรงกับบทบาทในระบบ

code-example.html

```
<!-- resources/views/layouts/app.blade.php -->
<!-- Right Side Of Navbar -->
<ul class="navbar-nav ms-auto">
  <!-- Authentication Links -->
  @guest
    @if (Route::has('login'))
      <li class="nav-item">
        <a class="nav-link" href="{{ route('login') }}">เข้าสู่ระบบ</a>
      </li>
    @endif

    @if (Route::has('register'))
      <li class="nav-item">
        <a class="nav-link" href="{{ route('register') }}">สมัครเป็นนักเขียน</a>
      </li>
    @endif
  @endguest
</ul>
```

การปรับแต่งแถบเมนูเมื่อล็อกอินสำเร็จ (@else ใน Layout)

- ในส่วน `@else` ของ Navbar (เมื่อผู้ใช้งานผ่านการตรวจสอบสิทธิ์แล้ว)
- เพิ่มข้อความทักทาย `สวัสดี , {{ Auth::user()->name }}` บนปุ่ม Dropdown
- แก้ไขข้อความในปุ่ม Logout ให้เป็น `ออกจากระบบ`
- เมื่อคลิก ออกจากระบบ จะทำการ Submit พอร์ม `logout-form` เพื่อทำลาย Session และออกจากระบบอย่างปลอดภัย

code-example.html

```
<!-- resources/views/layouts/app.blade.php -->
@else
    <li class="nav-item dropdown">
        <a id="navbarDropdown" class="nav-link dropdown-toggle" href="#" role="button"
            สวัสดี , {{ Auth::user()->name }}
        </a>

        <div class="dropdown-menu dropdown-menu-end" aria-labelledby="navbarDropdown">
            <a class="dropdown-item" href="{{ route('logout') }}"
                onclick="event.preventDefault();
                    document.getElementById('logout-form').submit();">
                ออกจากระบบ
            </a>

            <form id="logout-form" action="{{ route('logout') }}" method="POST"
                @csrf
```

การเพิ่มเมนูการจัดการบทความใน Dropdown (`เขียนบทความ` & `บทความของฉัน`)

- เพิ่มลิงก์เมนู **เขียนบทความ** ซึ่ไปยังเส้นทาง `href="/create"` เพื่อเปิดฟอร์มเขียนบทความ
- เพิ่มลิงก์เมนู **บทความของฉัน** ซึ่ไปยังเส้นทาง `href="/blog"` เพื่อดูรายการบทความทั้งหมด
- วางไว้ภายใน `<div class="dropdown-menu">` ด้านบนปุ่มออกจากระบบ
- ช่วยให้นักเขียนสามารถเข้าถึงเมนูสร้างและจัดการบทความได้อย่างสะดวกหลังล็อกอิน

code-example.html

```
<!-- resources/views/layouts/app.blade.php -->
<div class="dropdown-menu dropdown-menu-end" aria-labelledby="navbarDropdown">
  <a class="dropdown-item" href="/create">เขียนบทความ</a>
  <a class="dropdown-item" href="/blog">บทความของฉัน</a>
  <a class="dropdown-item" href="{{ route('logout') }}"
    onclick="event.preventDefault();
      document.getElementById('logout-form').submit();">
    ออกจากระบบ
  </a>

  <form id="logout-form" action="{{ route('logout') }}" method="POST" class="d
    @csrf
  </form>
</div>
```

การสืบทอด Layout ในหน้า View (`@extends('layouts.app')`)

- เปิดไฟล์หน้า View ที่ต้องการใช้งาน (เช่น `resources/views/blog.blade.php`)
- กำหนดคำสั่ง `@extends('layouts.app')` ไว้ที่บรรทัดบนสุดของไฟล์
- เป็นการสั่งให้หน้านี้ใช้งานโครงสร้าง Master Layout ร่วมกับระบบ Authentication
- หน้า View จะได้รับ Navbar, การตรวจสอบสิทธิ์ (@auth / @guest) และสไลด์ Bootstrap กันที

code-example.html

```
<!-- resources/views/blog.blade.php -->  
@extends('layouts.app')
```


สัปดาห์ที่ 12 · ระบบยืนยันตัวตนและการเข้าถึง

01-406-093-203 LARAVEL COURSE

ตั้งค่า Middleware (Authentication)

อาจารย์ผู้บรรยาย: ศุภมิตร จันทโรวงค์

สไลด์ที่ 17 / 23

การเรียกใช้ Middleware ใน Controller ('HomeController.php')

- เปิดไฟล์ `app/Http/Controllers/HomeController.php`
- กำหนด `$this->middleware('auth');` ในฟังก์ชัน `__construct()`
- คำสั่งนี้จะดักจับและบังคับให้ผู้ใช้ต้องล็อกอินก่อน จึงจะสามารถเรียกใช้งาน Method ภายใน Controller นี้ได้ (เช่น ฟังก์ชัน `index()` แสดงหน้า `home`)
- หากยังไม่ได้ล็อกอิน ระบบจะ Redirect ไปยังหน้า Login ให้อัตโนมัติ

code-example.php

```
// app/Http/Controllers/HomeController.php
public function __construct()
{
    $this->middleware('auth');
}

public function index()
{
    return view('home');
}
```

การป้องกันการเข้าถึงใน Controller หลัก (`AdminController.php`)

- เปิดไฟล์ `app/Http/Controllers/AdminController.php`
- เพิ่มฟังก์ชัน Constructor `public function __construct()`
- ใส่คำสั่ง `$this->middleware('auth');` ภายใน Constructor
- ช่วยป้องกันทุกฟังก์ชันใน AdminController (เช่น `blog()` , `about()` , `create()`) ไม่ให้ผู้ใช้ภายนอกเข้าถึงได้โดยไม่ล็อกอิน

code-example.php

```
// app/Http/Controllers/AdminController.php
public function __construct()
{
    $this->middleware('auth');
}
```

แบบทดสอบหลังเรียน (Post-test)

1. คำสั่ง Composer ใดที่ใช้ในการติดตั้งแพ็คเกจสำหรับสร้างระบบ Authentication พื้นฐานใน Laravel?

A. `composer require laravel/ui`

B. `composer require laravel/auth`

C. `composer install laravel/login`

D. `php artisan make:auth`

3. หลังจากรันคำสั่งสร้าง Scaffolding ของ UI แล้ว ต้องรันคำสั่งใดเพื่อดาวนโหลด Node Dependencies (เช่น Bootstrap และ Sass)?

A. `npm install`

B. `composer update`

C. `npm start`

D. `php artisan migrate`

5. ไฟล์ Master Layout หลักที่ระบบ Authentication สร้างขึ้นให้อัตโนมัติถูกจัดเก็บไว้ที่พาทใด?

A. `resources/views/layouts/app.blade.php`

B. `resources/views/master.blade.php`

C. `app/Views/layouts/main.blade.php`

D. `public/views/app.blade.php`

7. ในส่วน Navbar ของ `layouts/app.blade.php` หากต้องการแสดงชื่อของผู้ใช้งานที่กำลังล็อกอินอยู่ ต้องใช้คำสั่งใด?

A. `{{ Auth::user()->name }}`

B. `{{ User::getName() }}`

C. `{{ Session::get('user') }}`

D. `{{ $user_name }}`

9. หน้า View ลูก (เช่น `blog.blade.php`) ต้องใช้คำสั่งใดที่บรรจุที่จบสุดเพื่อสืบทอดโครงสร้างและ Navbar จาก `app.blade.php`?

A. `@extends('layouts.app')`

B. `@include('layouts.app')`

C. `@import('layouts.app')`

D. `@use('layouts.app')`

2. คำสั่ง Artisan ใดที่ใช้สร้างหน้าจอ Login, Register และ Controller ที่เกี่ยวข้องกับ Bootstrap?

A. `php artisan ui bootstrap --auth`

B. `php artisan make:auth --bootstrap`

C. `php artisan auth:scaffold bootstrap`

D. `php artisan ui:auth --bootstrap`

4. คำสั่งใดที่ใช้ให้ Vite ทำการคอมไพล์ไฟล์ SCSS, CSS และ JavaScript สำหรับการใช้งานจริง (Production Build)?

A. `npm run build`

B. `npm run dev`

C. `npm compile`

D. `npm run prod`

6. ใน Blade Template แกกคำสั่งใดที่ใช้ครอบการแสดงผลเนื้อหาเฉพาะสำหรับผู้ใช้งานที่ยังไม่ได้ล็อกอิน (Guest)?

A. `@guest ... @endguest`

B. `@auth ... @endauth`

C. `@user ... @enduser`

D. `@visitor ... @endvisitor`

8. หากต้องการให้แต่ละหน้า View สามารถกำหนดชื่อ Title ของตนเองเพื่อแสดงบนแท็บบน Browser ได้ ใน `layouts/app.blade.php` ควรกำหนดอย่างไร?

A. `<title>@yield('title')</title>`

B. `<title>{{ $title }}</title>`

C. `<title>@include('title')</title>`

D. `<title>@section('title')</title>`

10. หากต้องการป้องกันไม่ให้ผู้ใช้ที่ยังไม่ได้ล็อกอินเข้าถึงทุก Method ใน Controller (เช่น `HomeController` หรือ `AdminController`) ต้องกำหนดคำสั่งใดใน `__construct()`?

A. `$this->middleware('auth');`

B. `$this->protect('login');`

C. `$this->checkAuth();`

D. `Auth::check();`

คำถามท้ายบทเรียน

คำสั่งใดใช้ในการสร้างระบบ Authentication Scaffolding ที่มาพร้อม Bootstrap View และ Controller ใน Laravel?

A. `php artisan ui bootstrap --auth`

B. `composer require auth`

C. `php artisan make:auth`

D. `php artisan create:login`

คำอธิบาย: คำสั่ง `php artisan ui bootstrap --auth` จะสร้างระบบยืนยันตัวตน (Login, Register, Password Reset) พร้อม Bootstrap Layout ให้อัตโนมัติ

Workshop 12: ติดตั้งระบบ Authentication ด้วย laravel/ui และปรับแต่ง Layout & Middleware

- 1 ติดตั้งแพ็คเกจ `composer require laravel/ui` และรัน `php artisan ui bootstrap --auth` เพื่อสร้างระบบ Login/Register
- 2 รับคำสั่ง `npm install` และ `npm run build` เพื่อติดตั้งและคอมไพล์ Bootstrap Assets ให้พร้อมใช้งาน
- 3 ปรับแต่งไฟล์ `resources/views/layouts/app.blade.php` โดยเพิ่ม `<title>@yield('title')</title>` และแก้ไขเมนู `@guest / @else` ให้มีปุ่ม 'เขียนบทความ' และ 'บทความของฉัน'
- 4 สืบทอดโครงสร้างในหน้า View ด้วย `@extends('layouts.app')` และตั้งค่า Middleware ปกป้อง Controller ด้วย `$this->middleware('auth');`

command_prompt.sh

```
// app/Http/Controllers/AdminController.php
public function __construct()
{
    $this->middleware('auth');
}
```

ใบงานที่ 12: การพัฒนาระบบ Authentication, ตกแต่ง Layout และควบคุมสิทธิ์ด้วย Middleware

ให้นักศึกษาติดตั้งระบบ Authentication แบบ Bootstrap ด้วย Laravel UI, ปรับแต่ง Master Layout (app.blade.php) ให้มีเมนูจัดการบทความ และเชื่อมต่อหน้า View ทั้งหมด พร้อมทั้งตั้งค่า Middleware ปกป้อง AdminController ให้เข้าถึงได้เฉพาะผู้ที่ล็อกอินแล้วเท่านั้น

เกณฑ์การตรวจประเมินผล:	
การติดตั้ง Laravel UI และคอมไพล์ Bootstrap Asset สำเร็จ	30%
การปรับแต่ง Master Layout	app.blade.php และสืบทอดในหน้า View สมบูรณ์
การตั้งค่า Middleware ดักจับสิทธิ์ใน Controller ได้ถูกต้อง	35%

